



UNIVERSITY OF
LIVERPOOL

Application Development with C++ (ELEC362)

Lecture 4: Non-sequential codes

mihasan@liverpool.ac.uk

Previous lecture

- The data types of variables, their declaration and initialisation methods were discussed.
- Operators in C++ and their precedence were discussed.
- Variable casting was discussed.
- Arrays declarations and initialisations were discussed.
- Further reading: <http://www.cplusplus.com/doc/tutorial/>

This lecture

- What is covered in this lecture?

Situations where the code does not follow the order of appearance

- Why is it covered?

Because it provides large flexibility in the flow of the programme.

- How are topics covered in this lecture:

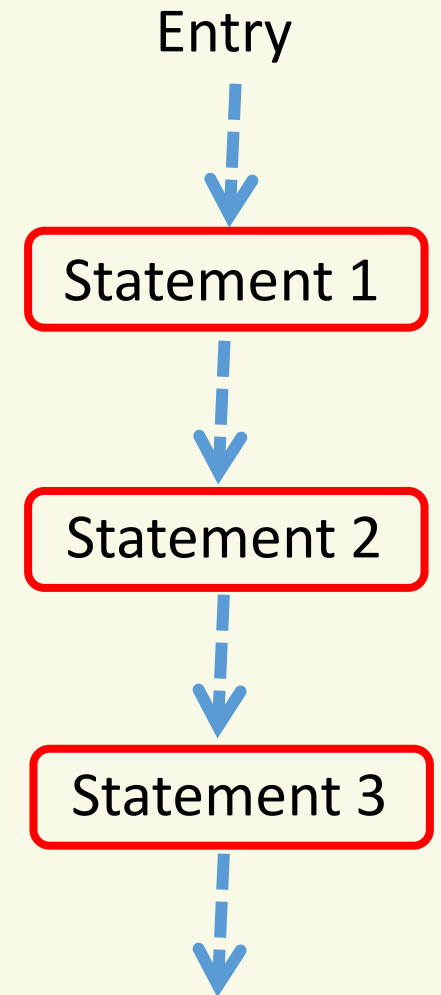
5 source codes and a practical example.

Sequential vs non-sequential codes

- Up until this point all the codes we have seen are *sequential* (i.e. executes one statement after another as they appear).
- There are three cases where the code runs non-sequentially:
 1. When using a control structure (*selection* or *repetition*)
 2. When calling a function
 3. When handling an error.

Practical note:

- When explaining algorithms always use flow charts.

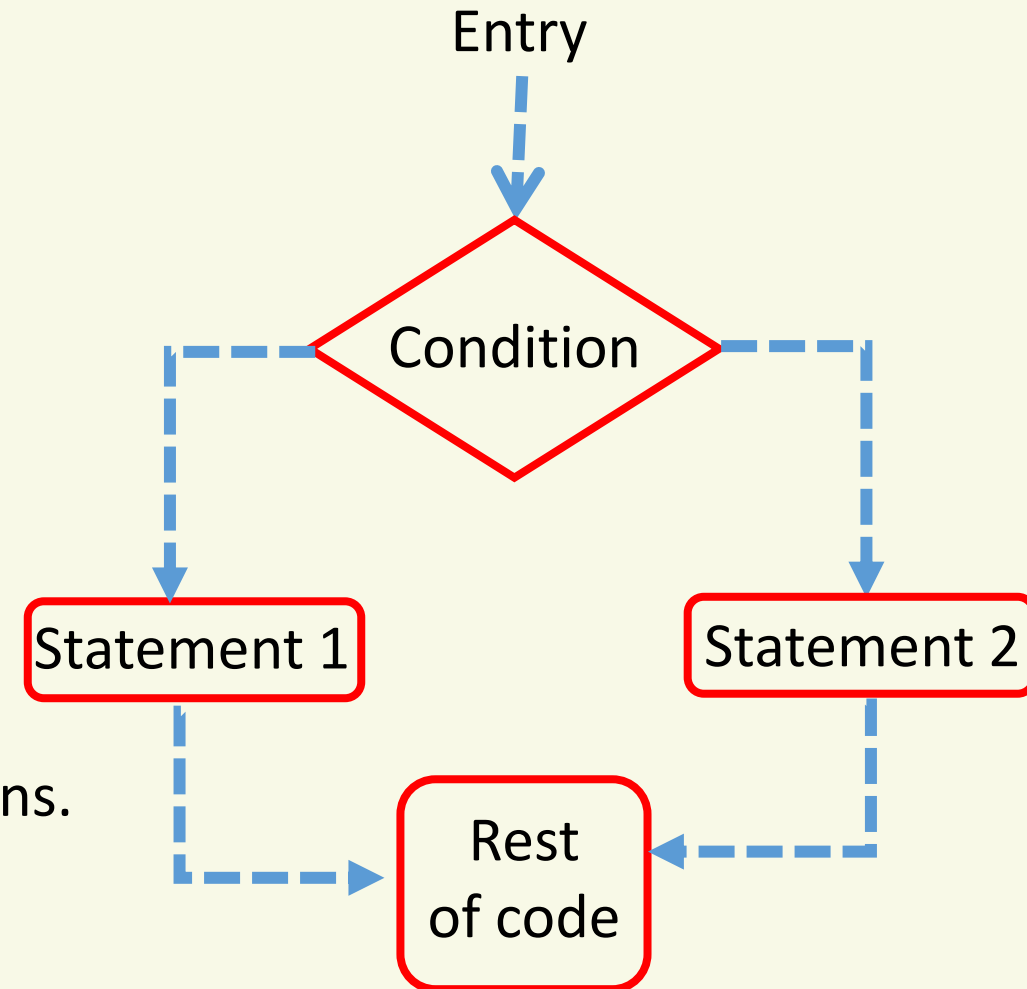


Selection structure : if-else

- The main selection structures in C++ are *if-else* and *switch* statements.
- The syntax of if-else structure is as follows:

```
if (condition)
{ //statements (condition is true);
}
else { //statements (condition is false);
}
```

- Conditions can be Boolean variables or expressions.



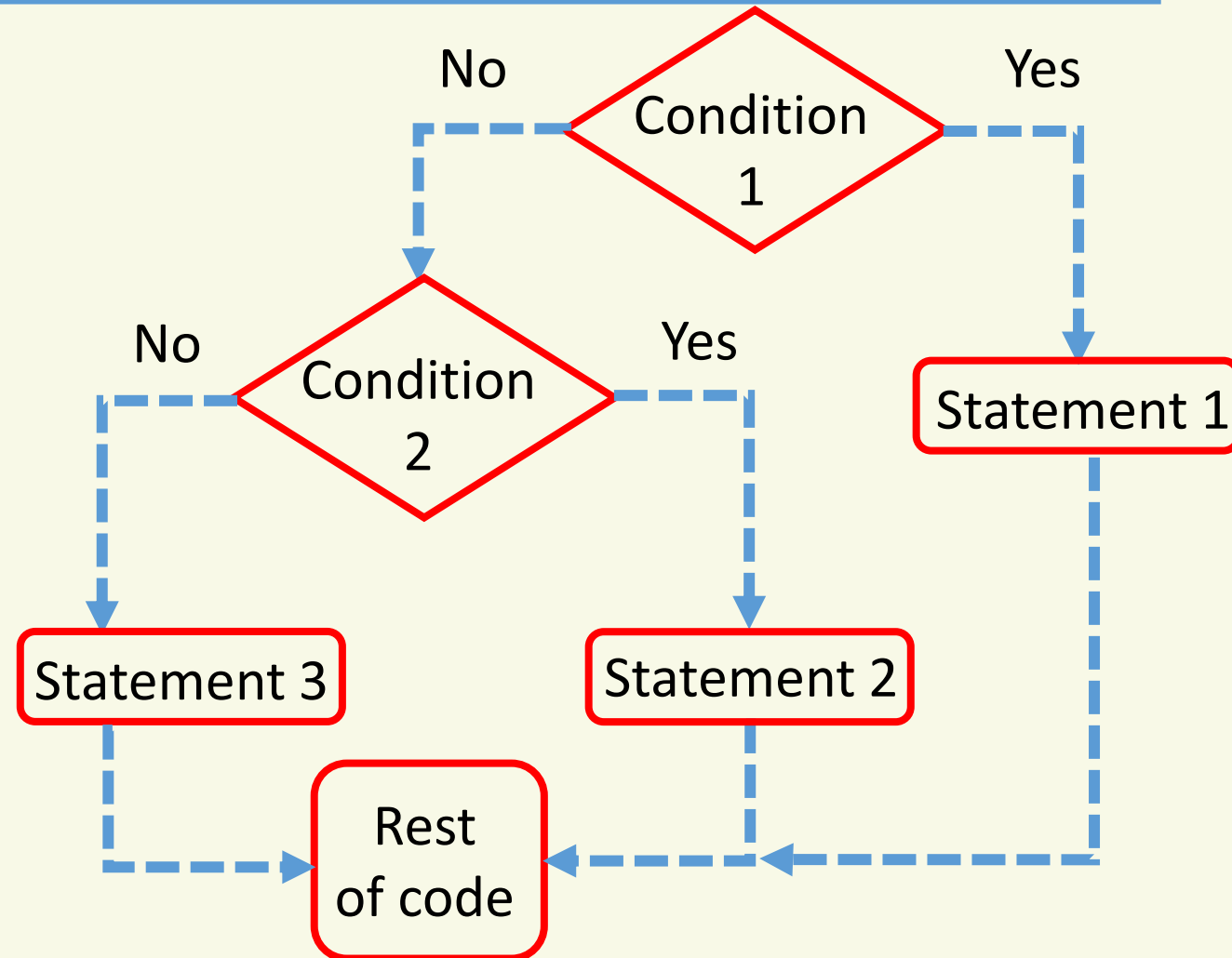
Selection structure: if-else

- *if-else* structure can be nested.

```
if (condition1)
{ //Statement1
}
else if (condition2)
{ //Statement2
}
else { //Statement3 }
```

- A compact way to write if-else is using the operator “?”:

```
Condition? If_true : If_false
```



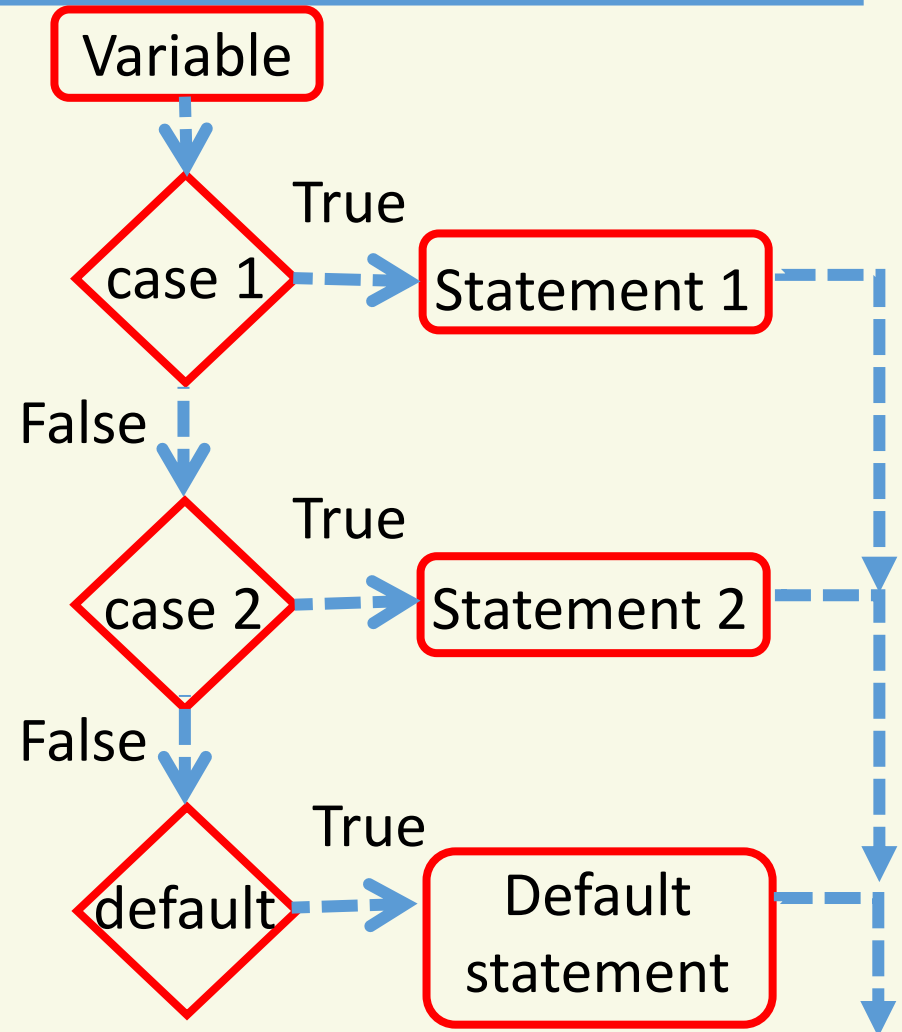
Selection structure: switch-case

- *Switch-case* is more efficient than nested *if-else*.
- The syntax of switch-case structure is as follows:

```
switch (variable)
{
    case 1:
        //do something
        break;
    case 2:
        // do something else
        break;
    default:
        // do this when out of options
        break;
}
```

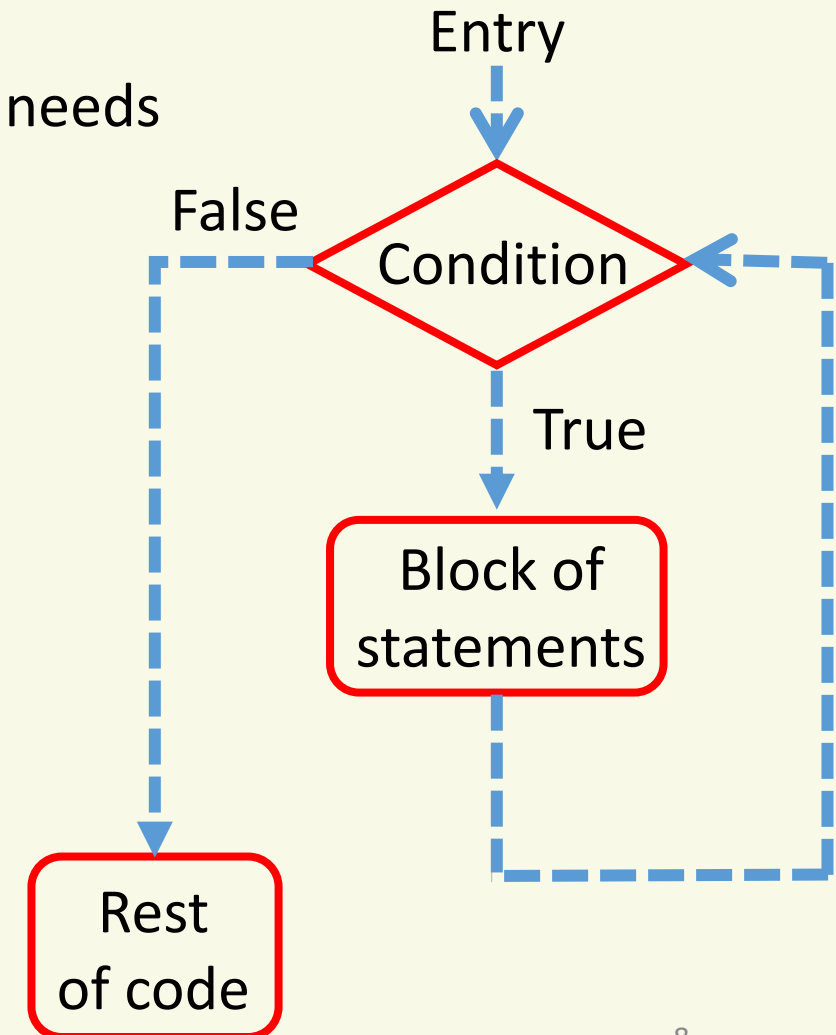
Can be a char or int datatypes only

- Go to [L4D1.cpp](#)



Repetition structure

- Repetition structures are used when a part of the code needs to be repeated.
- The most common repetition structures in C++ are:
 1. For loops: controlled by the number of runs.
 2. While loops: controlled by a condition.
 3. Do-while loops: a modified version of a while loop.



For loop

- Syntax:

```
for (unsigned int counter = 0; counter < N; counter++)  
{ // statements;  
}
```

Counter variable initialisation

Condition checking Counter handling

- Mechanism:
 1. Loop counter is initialised.
 2. Condition is checked.
 3. If true, Statements in the loop's body are executed.
 4. Counter handling.
 5. Repeat steps 2 to 5.

Management of For loops

- The execution of a For loop can be altered by using **break** , **continue**, or **goto**.

Statement	functionality
break	Terminates the loop even if the condition on the loop counter is true
continue	Starts a new iteration in the loop
goto	Can be used to jump from any part of the code to another

- Statements including **break** and **goto** can appear anywhere in the code (not necessarily in a loop).

Practical note:

- **goto** is a depreciated statement and is no longer in use by professional C++ developers.

Management of For loops

- Example:

```
for (unsigned int counter = 0; counter < 10; counter++)  
{  
  
    if ( counter == 5) continue; // Skips 5  
  
    if (counter > 6 ) goto Last; //Moves execution to "Last" statement  
  
    if (counter == 9) break;  
  
Last: std::cout << counter <<endl;  
}
```

While loop and Do-while loop

- Syntax:

```
while (condition)
{ // statements;
}
```

Boolean
expressions

```
do { // statements;
}
while (condition)
```

While loop Mechanism:

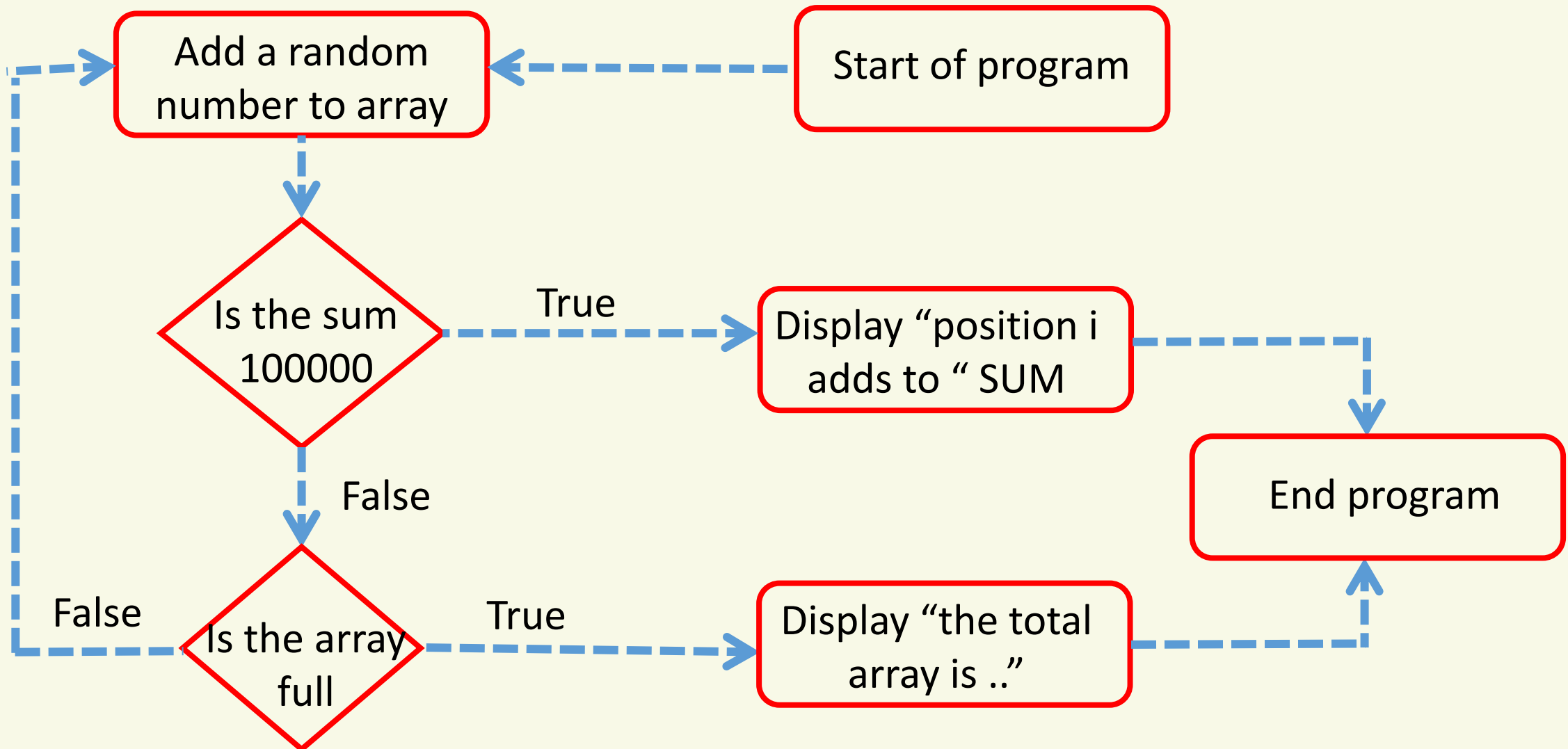
1. Check condition.
2. If true, execute statements.
3. Repeat steps 1-2.

Do-while loop Mechanism:

1. Execute statements.
2. Check condition.
3. If true, Repeat steps 1-2.

- Remember to control the condition from within the loop to avoid infinite loop.
- Go to [L4D2.cpp](#)

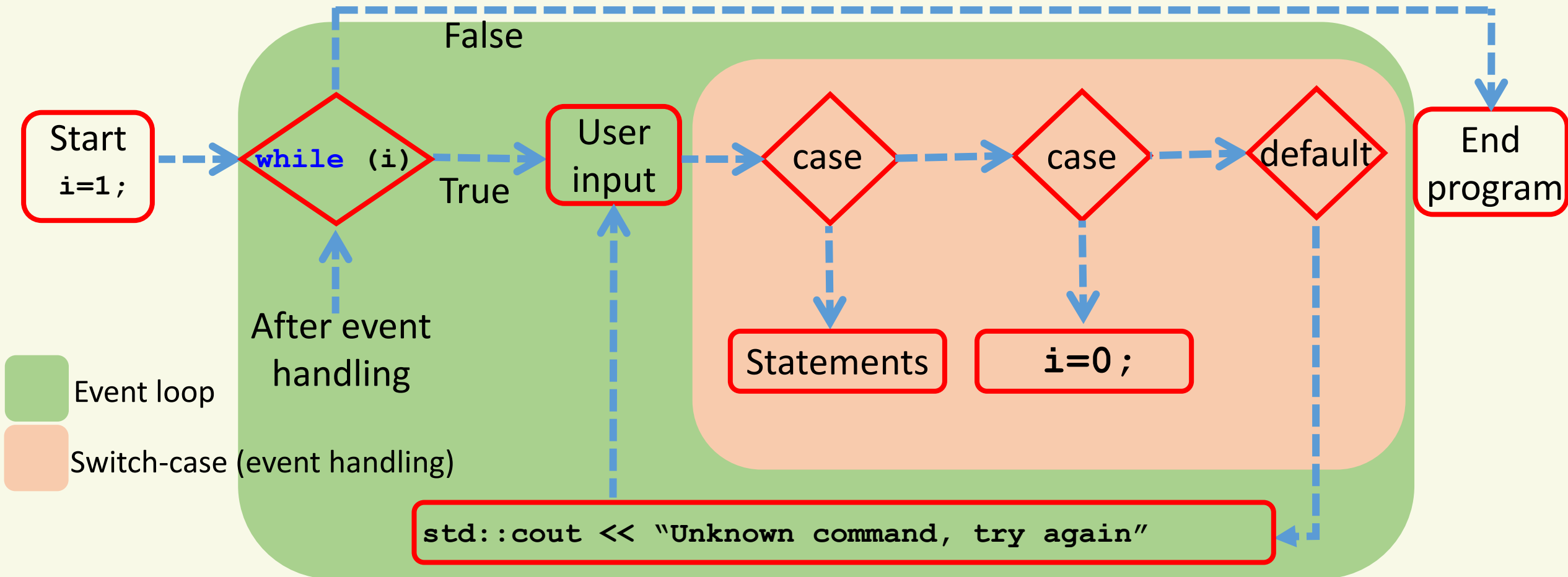
How L4D2 works



Practical Example: Event loop

- The core of any programme is a code structure known as “Event loop”.
- When the programme starts, an infinite loop is initialised.
- The body of the loop waits for the user’s input (events).
- When users input something, their input is dealt with by a switch-case structure.
- For every possible input, there is one case in the switch-case structure that handles the input (this is known as event handler).
- One of the cases is assigned to terminate the loop (i.e. close the programme).
- Example :cmd.exe (in Windows).
- Go to [L4D5.cpp](#)

Practical Example: Event loop



Practical note: Implement an event loop for any console based application you develop.

Functions in C++

- A function is a group of statements that together perform a defined task.
- Syntax:

```
return_type FunctionName (datatype argument1, ..., ...)  
    { // Statements;  
        return output; }
```

- Naming convention follows Google C++ style code (every word starts with capital).
- Few rules:
 1. Functions has to be declared before main function.
 2. Every function only returns one value.
 3. The datatype of the output has to match the return type of the function.
 4. Return type of `void` is used when the function has no return.

Functions declarations and arguments passing

- There are two styles of declaring a function (with or without function declaration):

```
int MyFun(int argument); //declaration

int main () {
    //Statements;
}

int MyFun(int argument) {
    //Statements;
}
```

```
int MyFun(int argument) {
    //Statements;
}

int main () {
    //Statements;
}
```

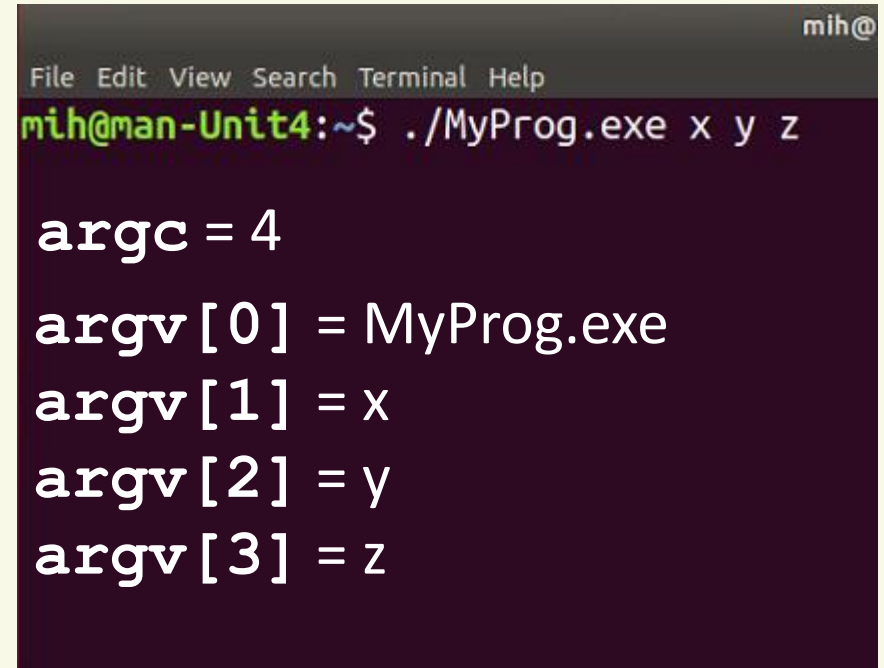
- It is more common in codes to follow function declaration style.
- Only one variable is allowed as a return, for more advanced options watch out for Lecture 5!!

Arguments of the `main` function

- It is possible to run programmes in batch mode (running a programme simply by calling it from a command line).
- In such context, the main function can have parameters, most commonly:

```
int main (int argc, char *argv[])
```

- **argc** is a variable containing the number of arguments passed to the programme through command line. **argv[]** is an array of character set, each set describe one word of the input sequence
- Go to [L4D4.cpp](#)

A terminal window with a dark background and light text. The title bar shows 'mih@'. The menu bar includes 'File', 'Edit', 'View', 'Search', 'Terminal', and 'Help'. The prompt is 'mih@man-Unit4:~\$'. The command entered is './MyProg.exe x y z'. Below the command, the program's output is displayed: 'argc = 4', 'argv[0] = MyProg.exe', 'argv[1] = x', 'argv[2] = y', and 'argv[3] = z'.

```
mih@  
File Edit View Search Terminal Help  
mih@man-Unit4:~$ ./MyProg.exe x y z  
  
argc = 4  
argv[0] = MyProg.exe  
argv[1] = x  
argv[2] = y  
argv[3] = z
```

Exceptions and error handling

- Exceptions are run-time anomalies or errors a program encounters during execution.
- Exception handling simply means telling the code what to do when an exception occurs.
- Exception handling can be done using three keywords:
 1. **try**: Defines the code block where the exception may occur (be thrown).
 2. **throw**: Used to declares the exception.
 3. **catch**: Defines the response of the program if a particular exception occurs.
- Go to [L4D5.cpp](#)

Practical note: • Handle errors using try-throw-catch structure in your code.
• Give numbers to exceptions and keep a table of what they mean.

Summary

- Cases where the code may not run sequentially were discussed
- If-else and switch-case structures were discussed under selection structures.
- For loop, While and Do-while loops were discussed under repetition structures.
- The concept of event loops was discussed.
- The definition of functions and how they work was discussed.
- Error handling was discussed.